

15

Лекция

15–16 декабря

Коды, исправляющие ошибки

Константин Чухарев

Дискретная математика — Осень 2026



Коды, исправляющие ошибки

| *Цель вычислений — проникновение в суть, а не числа.*

— *Ричард Хэмминг*

Канал шумит

Канал передачи данных искажает биты.

Провод, радиоволна, дорожка диска — всё это каналы, и все они ошибаются.

Запись на диск — тоже передача, только из прошлого в будущее.

- В памяти с коррекцией бит может перевернуться от космической частицы.
- Царапина на диске портит дорожку не по одному биту, а целым участком.

Защита и сжатие

- **Защита:** добавить избыточность, чтобы исправить ошибки.
- **Сжатие:** убрать избыточность, чтобы уменьшить размер.

Коды, исправляющие ошибки, работают в QR-кодах, на дисках и в спутниковой связи.

Сжатие без потерь живёт в zip и PNG.

Плата за защиту — полезная скорость: чем больше избыточности, тем меньше данных проходит через канал за единицу времени.

Алфавит и слова

Определение 1 (Алфавит)

Алфавит — конечное непустое множество символов.

Например, двоичный алфавит $\{0, 1\}$ или алфавит ДНК $\{A, C, G, T\}$.

Определение 2 (Слово)

Слово над алфавитом Σ — конечная последовательность символов из Σ .

Число символов называется **длиной** слова.

Слово 10110 над $\{0, 1\}$ имеет длину 5, а двоичных слов длины n ровно 2^n .

Код и кодирование

Определение 3 (Код)

Код — непустое множество слов, которые называются **кодowymi словами**.

Определение 4 (Кодирование)

Кодирование — инъективная функция $E : M \rightarrow C$ из множества сообщений M в код C .

Декодирование возвращает сообщение из кодового слова: $D(E(m)) = m$.

Из инъективности следует $|M| \leq |C|$: у каждого сообщения своё слово.

Кодирование без инъекции

Пример

Отображение сообщений A, B, C в слова:

$$E(A) = 0, \quad E(B) = 1, \quad E(C) = 0.$$

Принятое слово 0 с равным успехом могло быть A или C .

Ответа нет: различие между сообщениями стёрто ещё до канала.

Дыра в кодировании не чинится хитрым декодером.

Если $E(A) = E(C)$, принятое слово сообщает лишь общее: пришло что-то одно из двух.

Цепочки сообщений

Поток передаётся кусками, и приёмник должен разрезать его обратно на сообщения.

Пример

Пусть $E(x) = 0$, $E(y) = 1$, $E(z) = 01$.

Каждое сообщение закодировано своим словом, но слово 01 читается как z и как пара x, y .

Инъективности на отдельных сообщениях мало: нужна однозначность разбиения потока.

Её дают префиксные коды.

Чётность: первая защита

Пример

К двум битам данных добавим третий — их XOR:

$$00 \mapsto 000, \quad 01 \mapsto 011, \quad 10 \mapsto 101, \quad 11 \mapsto 110.$$

Передали 00, канал перевернул второй бит: принято 010, единиц одна — чётность нарушена, ошибку видно.

Но какой бит испорчен, не узнать: от принятого слова три кодовых слова отличаются ровно одним битом.

Две ошибки дают слово с чётным числом единиц: из 00 выходит 110 — кодовое слово, и код молчит.

Код чётности с $d = 2$ обнаруживает одну ошибку и не исправляет ни одной.

Код-повторение

Пример

Код-повторение передаёт каждый бит трижды: слова 000 и 111.

Принято 011: единиц две против одного нуля — голосование по большинству возвращает 1.

Одна ошибка исправляется, но два из трёх битов канала потрачены на защиту, а не на данные.

Защита оплачивается скоростью.

Код-повторение сжигает две трети канала, чтобы исправить один бит.

Расстояние Хэмминга

Определение 5 (Расстояние Хэмминга)

Расстояние Хэмминга — число позиций, в которых строки различаются:

$$d(x, y) = |\{i \mid x_i \neq y_i\}|.$$

Пример

$d(10110, 10001) = 3$: различаются позиции 3, 4 и 5.

$d(0000, 1111) = 4$, $d(x, x) = 0$.

Метрика

Расстояние Хэмминга — метрика на строках.

- Оно неотрицательно и равно нулю только для совпадающих строк.
- Оно симметрично: $d(x, y) = d(y, x)$.
- Выполняется неравенство треугольника: $d(x, z) \leq d(x, y) + d(y, z)$.

Чтобы превратить x в y , нужно перевернуть ровно $d(x, y)$ битов.

Неравенство треугольника гарантирует: ошибки канала не сблизят два кодовых слова.

Вес слова

Определение 6 (Вес слова)

Весом слова x называется число его единичных позиций, обозначаемое $\text{wt}(x)$.

Пример

Единицы слова 10110 стоят в позициях 1, 3 и 4 — отсюда вес 3.

$$\text{wt}(111) = 3, \text{wt}(000) = 0.$$

Расстояние и вес связаны: $d(x, y) = \text{wt}(x + y)$, где $+$ — побитовый XOR.

Разошедшиеся позиции — это ровно единицы суммы $x + y$.

Расстояние: полный расчёт

Считаем расстояние по-честному, позиция за позицией.

i	1	2	3	4	5	6	7	8
x	1	0	1	1	0	1	0	0
y	1	0	0	1	1	1	1	0
$x + y$	0	0	1	0	1	0	1	0

Разошлись позиции 3, 5 и 7 — три единицы в сумме.

$$d(x, y) = \text{wt}(x + y) = 3.$$

Кодовое расстояние

Определение 7 (Кодовое расстояние)

Кодовое расстояние — минимум расстояний между разными кодовыми словами:

$$d = \min(\{d(x, y) \mid x, y \in C, x \neq y\}).$$

Пример

Код-повторение $\{000, 111\}$: пара слов всего одна, и она разошлась в трёх позициях — $d = 3$.

Код чётности $\{000, 011, 101, 110\}$: слова 011 и 101 различаются в двух позициях, и ближе пары нет — $d = 2$.

Параметры кода

Определение 8 (Параметры кода)

Код характеризуется тройкой $(n, |C|, d)$: длина, число слов, кодовое расстояние.

Для линейных кодов пишут (n, k, d) , где k — число информационных битов, и тогда $|C| = 2^k$.

Пример

Код Хэмминга: $(7, 4, 3)$ — четыре бита данных, $2^4 = 16$ слов, расстояние 3.

Код-повторение: $(3, 1, 3)$.

Скорость и избыточность

Определение 9 (Скорость кода)

Скорость $R = \frac{k}{n}$ — доля информационных битов.

Избыточность $1 - R$ — доля служебных.

Код	Скорость	Исправляет
Повторение (3, 1)	$1/3 \approx 0.33$	1 бит
Хэмминг (7, 4)	$4/7 \approx 0.57$	1 бит

При одинаковой надёжности код Хэмминга передаёт примерно в $\frac{12}{7} \approx 1.7$ раза больше данных, чем повторение.

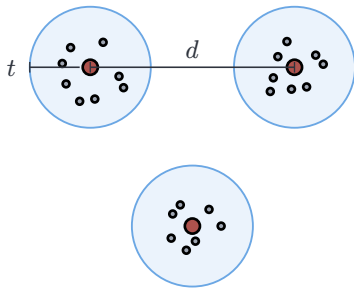
Повторение — простая и дорогая форма избыточности.

Шары вокруг слов

Кодовые слова — точки в пространстве всех двоичных строк длины n .

Декодер заменяет принятую строку ближайшим кодовым словом — как текстовый редактор исправляет опечатку по ближайшему правильному слову.

Каждое слово окружено шаром радиуса t : все строки внутри шара декодируются в его центр.



Способность кода

Теорема 1 (Способность кода)

Если $d \geq s + 1$, код обнаруживает до s ошибок.

Если $d \geq 2t + 1$, код исправляет до t ошибок, то есть

$$t = \left\lfloor \frac{d - 1}{2} \right\rfloor.$$

Пример

Код с $d = 3$ исправляет одну ошибку.

Код с $d = 4$ исправляет по-прежнему одну, зато отличает двойную ошибку от одиночной.

Декодирование по ближайшему

Определение 10 (Декодирование по ближайшему)

Принятое слово r декодируется в кодовое слово c , для которого $d(r, c)$ минимально.

Пока ошибок не больше t , ближайшее слово — единственное и правильное.

При большем числе ошибок декодер всё равно вернёт ближайшее, но это уже совпадение, а не гарантия.

Доказательство способности кода

Доказательство

Докажем оба утверждения по очереди.

Обнаружение. Пусть $d \geq s + 1$, и принятое слово отличается от переданного не более чем в s позициях. Тогда оно отстоит от переданного слова меньше, чем на d , поэтому не может совпасть с другим кодовым словом, — ошибка обнаружена.

Исправление. Пусть теперь $d \geq 2t + 1$. Шары радиуса t вокруг кодовых слов не пересекаются: если бы точка z лежала в двух шарах с центрами x и y , получилось бы $d(x, y) \leq d(x, z) + d(z, y) \leq 2t$ — противоречие.

Принятое слово с не более чем t ошибками попадает ровно в один шар и декодируется в его центр.

Почему не $\lfloor \frac{d}{2} \rfloor$?

Считать способность кода по формуле $t = \lfloor \frac{d}{2} \rfloor$ — типичная ошибка.

Код $\{0000, 1111\}$ имеет $d = 4$, и формула обещает $t = 2$.

Но слово 0011 отстоит от обоих кодовых слов ровно на 2: декодеру нечего выбрать.

Верный счёт: $t = \lfloor \frac{d-1}{2} \rfloor = 1$.

Код Хэмминга (7, 4)

Определение 11 (Код Хэмминга)

Параметры: $n = 7$, $k = 4$ бита данных, $d = 3$.

Проверочные биты занимают позиции 1, 2, 4 — степени двойки, данные стоят на позициях 3, 5, 6, 7:

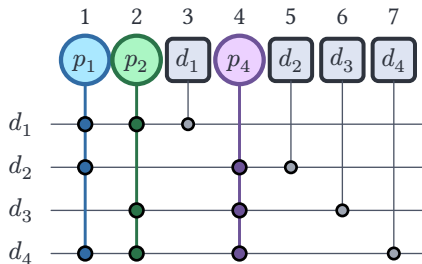
$$p_1, p_2, d_1, p_4, d_2, d_3, d_4.$$

Хэмминг придумал код от раздражения: машина с перфокартами останавливалась на каждой ошибке чтения, и вычисления пропадали впустую.

Его статья 1950 года дала первый явный код, исправляющий одну ошибку.

Контрольные группы

Проверочный бит — XOR данных своей группы.



- p_1 контролирует d_1 , d_2 , d_4 .
- p_2 контролирует d_1 , d_3 , d_4 .
- p_4 контролирует d_2 , d_3 , d_4 .

Почему такие группы?

Бит p_i контролирует позиции, в двоичной записи которых i -й бит равен 1.

Позиция $3 = 011$ входит в первые две группы, позиция $7 = 111$ — во все три.

Три проверки работают как вопросы с ответом «да-нет»: каждая делит позиции пополам и отдаёт один бит номера позиции.

Кодирование

Пример

Данные $d_1 = 1, d_2 = 0, d_3 = 1, d_4 = 1$.

$$p_1 = 1 \oplus 0 \oplus 1 = 0, \quad p_2 = 1 \oplus 1 \oplus 1 = 1, \quad p_4 = 0 \oplus 1 \oplus 1 = 0.$$

Кодовое слово: $(0, 1, 1, 0, 0, 1, 1)$.

Каждая контрольная группа в слове 0110011 даёт нулевой XOR — слово кодовое.

Каждый проверочный бит — один XOR своей группы: три бита защиты обходятся в три операции над четырьмя битами данных.

Синдром

Определение 12 (Синдром)

При приёме каждый бит синдрома — XOR своей контрольной группы:

$$s_1 = p_1 \oplus d_1 \oplus d_2 \oplus d_4, \quad s_2 = p_2 \oplus d_1 \oplus d_3 \oplus d_4, \quad s_4 = p_4 \oplus d_2 \oplus d_3 \oplus d_4.$$

Нулевой синдром означает, что ошибок нет.

Ненулевой синдром читается как двоичное число $(s_4 s_2 s_1)_2$.

Оно равно номеру искажённой позиции — надо лишь перевернуть этот бит.

Таблица синдромов

s_1	s_2	s_4	$(s_4 s_2 s_1)_2$	Позиция
1	0	0	1	1
0	1	0	2	2
1	1	0	3	3
0	0	1	4	4
1	0	1	5	5
0	1	1	6	6
1	1	1	7	7

Совпадение синдрома с номером позиции — следствие выбора контрольных групп.

Столбцы проверочной матрицы — двоичные записи чисел от 1 до 7, поэтому синдром складывает столбцы искажённых позиций.

Исправление по синдрому

Пример

Передано $(0, 1, 1, 0, 0, 1, 1)$, ошибка в позиции 5: принято $(0, 1, 1, 0, 1, 1, 1)$.

$$s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1, \quad s_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0, \quad s_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1.$$

Синдром $(1, 0, 1) = 5$: переворачиваем пятый бит — слово $(0, 1, 1, 0, 0, 1, 1)$ и данные $(1, 0, 1, 1)$ вернулись.

Кодирование и декодирование зеркальны: те же три группы, тот же XOR.

Проверочная матрица

Три контрольные группы собираются в одну матрицу:

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Слово кодовое, если $Hx^T = 0$ над $\text{GF}(2)$.

Пример

Для слова $(0, 1, 1, 0, 0, 1, 1)$ проверка даёт $Hx^T = (0, 0, 0)^T$.

Для принятого слова синдром — это Hr : он называет позицию ошибки.

Для длины $2^l - 1$ берут все ненулевые столбцы высоты l : нулевой столбец пропустил бы ошибку в своей позиции.

Две ошибки

Пример

Передано $(0, 1, 1, 0, 0, 1, 1)$, ошибки в позициях 4 и 6: принято $(0, 1, 1, 1, 0, 0, 1)$.

Синдром $(0, 1, 0)$ имеет значение 2.

Декодер переворачивает бит 2 — неиспорченную позицию.

Две ошибки маскируются под одну.

Код с $d = 3$ их замечает, но исправить не может.

Три ошибки

Пример

Ошибки в позициях 1, 2 и 3: вектор ошибок $(1, 1, 1, 0, 0, 0, 0)$ сам является кодовым словом.

Принято $(1, 0, 0, 0, 0, 1, 1)$ — снова кодовое слово. Синдром нулевой: декодер сообщает «ошибок нет», хотя данные испорчены.

За пределами $t = \lfloor \frac{d-1}{2} \rfloor$ исправлений код за результат не отвечает.

Расширенный код (8, 4, 4)

К базовому коду добавляется один общий бит чётности всего слова.

Почему расстояние вырастает до 4: слова базового кода различаются минимум в трёх позициях, и если различий ровно три, их нечётность делает разными биты чётности — вот четвёртая позиция.

Теорема 2 (Что даёт бит чётности)

Код (8, 4, 4) по-прежнему исправляет одну ошибку.

Зато он отличает двойную ошибку от одиночной: двойная сохраняет чётность слова, одиночная — нет.

Четыре исхода декодера

Синдром базового кода плюс общий бит чётности различают четыре ситуации:

Синдром	Чётность	Что делать
0	сошлась	ошибок нет
0	не сошлась	испорчен бит чётности — исправляем
не 0	не сошлась	одна ошибка — исправляем
не 0	сошлась	две ошибки — не трогаем

При подозрении на две ошибки декодер сообщает о повреждении данных, вместо того чтобы исправлять не тот бит.

Две ошибки в расширенном коде

Пример

Передано $(0, 1, 1, 0, 0, 1, 1, 0)$, данные $(1, 0, 1, 1)$.

Ошибки в позициях 4 и 6: принято $(0, 1, 1, 1, 0, 0, 1, 0)$.

Синдром первых семи бит — $(0, 1, 0) = 2$, а чётность всего слова сошлась: единиц четыре.

Бит 2 декодер не трогает: синдром указывает на неиспорченную позицию, и исправление было бы ложным — слово объявляется повреждённым.

Граница Хэмминга

Упаковка шаров ограничивает число кодовых слов сверху.

Теорема 3 (Граница Хэмминга)

Двоичный код длины n , исправляющий t ошибок, содержит не более

$$|C| \leq \frac{2^n}{\sum_{i=0}^t \binom{n}{i}}$$

СЛОВ.

Каждому кодовому слову нужен свой шар, а пространство не резиновое.

Пример (Граница в числах)

При $n = 7$, $t = 1$ шар вмещает $1 + 7 = 8$ строк, и граница даёт $|C| \leq \frac{128}{8} = 16$.

Код $(7, 5, 3)$ из 32 слов запрещён: $2^5 \cdot 8 = 256 > 128$.

Доказательство границы Хэмминга

Доказательство

Докажем подсчётом точек.

Код исправляет t ошибок, поэтому его кодовое расстояние $\geq 2t + 1$, и шары радиуса t вокруг кодовых слов попарно не пересекаются: иначе два центра оказались бы на расстоянии не больше $2t$.

На расстоянии ровно i от данного слова лежит $\binom{n}{i}$ строк — столько есть способов выбрать искажённые позиции.

Значит, шар радиуса t вмещает ровно $\sum_{i=0}^t \binom{n}{i}$ строк.

Непересекающиеся шары лежат внутри пространства из 2^n строк, поэтому $|C| \cdot$

$$\sum_{i=0}^t \binom{n}{i} \leq 2^n.$$

Совершенный код

Определение 13 (Совершенный код)

Код называется **совершенным**, если шары радиуса t вокруг его слов покрывают всё пространство без пересечений и без дыр.

Для него граница Хэмминга обращается в равенство.

Пример

Код Хэмминга $(7, 4)$ исправляет одну ошибку, и

$$2^4 \cdot \left(1 + \binom{7}{1}\right) = 16 \cdot 8 = 128 = 2^7.$$

Восемь значений синдрома — семь позиций ошибок и чистый приём.

Код-повторение $(3, 1, 3)$ совершенный по той же причине: $2 \cdot (1 + 3) = 8 = 2^3$.

Граница Синглтона

Теорема 4 (Граница Синглтона)

Код длины n с 2^k словами и кодовым расстоянием d удовлетворяет неравенству

$$k \leq n - d + 1.$$

Доказательство

Докажем вычёркиванием позиций.

Удалим из каждого кодового слова одни и те же $d - 1$ позиций.

Укороченные слова все различны: два совпавших исходных слова расходились бы не более чем в $d - 1$ позициях, а кодовые слова отстоят друг от друга минимум на d .

Строк длины $n - d + 1$ ровно 2^{n-d+1} , поэтому $2^k \leq 2^{n-d+1}$.

Коды MDS

Определение 14 (Код MDS)

Код называется **кодом MDS** (от англ. maximum distance separable), если он достигает границы Синглтона: $d = n - k + 1$.

Считаем $n - k + 1$ для знакомых кодов и сравниваем с d :

Код	(n, k, d)	$n - k + 1$	MDS?
Повторение	$(3, 1, 3)$	3	да
Чётность	$(n, n - 1, 2)$	2	да
Хэмминг	$(7, 4, 3)$	4	нет
Расширенный Хэмминг	$(8, 4, 4)$	5	нет

Почему коды Хэмминга не MDS?

Оба кода Хэмминга отстают от границы на одну позицию расстояния: при $k = 4$ граница разрешает $d \leq 4$ на длине 7 и $d \leq 5$ на длине 8.

Над $\text{GF}(2)$ границу Синглтона достигают только простые коды — повторение и чётность.

Над большими алфавитами границу держат коды Рида–Соломона, на которых работают QR-коды и компакт-диски.

Линейные коды

Определение 15 (Линейный код)

Множество $C \subseteq \text{GF}(2)^n$ называется **линейным кодом**, если оно содержит нулевое слово и замкнуто относительно покомпонентного XOR.

Сумма двух кодовых слов — снова кодовое слово.

Все двоичные строки длины n — векторное пространство $\text{GF}(2)^n$ над полем из двух элементов: XOR — сложение, AND — умножение.

Базис из k строк порождает ровно 2^k слов: разные подмножества строк дают разные суммы.

Поэтому у линейного кода $|C| = 2^k$, и параметры пишутся (n, k, d) .

Порождающая матрица

Определение 16 (Порождающая матрица)

Матрица G размера $k \times n$, строки которой образуют базис кода, называется **порождающей**.

Кодирование сообщения $m \in \text{GF}(2)^k$ — умножение $c = mG$.

Пример

Для кода Хэмминга:

$$G = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Сообщение $(1, 0, 1, 1)$: складываем строки 1, 3 и 4 — получаем $(0, 1, 1, 0, 0, 1, 1)$.

Расстояние линейного кода

Теорема 5 (Кодовое расстояние линейного кода)

$$d = \min(\{\text{wt}(c) \mid c \in C, c \neq 0\}).$$

Доказательство

Докажем двумя неравенствами.

Пусть c — ненулевое слово наименьшего веса. Нулевое слово принадлежит коду, и пара c и 0 даёт $d \leq \text{wt}(c)$.

Обратно, для любых разных $x, y \in C$ сумма $x + y$ — ненулевое кодовое слово, и $\text{wt}(x + y) = d(x, y)$, поэтому d не меньше веса наименьшего ненулевого слова.

Перебор сокращается с $\binom{16}{2} = 120$ пар до 15 ненулевых слов.

Веса слов кода Хэмминга

Теорема 6

Кодовое расстояние кода Хэмминга $(7, 4)$ равно 3.

Доказательство

Докажем через минимальный вес: решения $Hx^T = 0$ образуют линейный код, а у линейного кода d равен весу наименьшего ненулевого слова.

Вес 1 невозможен: единица в позиции i оставляет в Hx^T столбец i , а все семь столбцов матрицы ненулевые.

Вес 2 невозможен: столбцы H попарно различны, поэтому сумма двух разных столбцов ненулевая.

Вес 3 достижим: $(1, 0, 0) + (0, 1, 0) + (1, 1, 0) = (0, 0, 0)$, поэтому слово 1110000 кодовое.

Код Хэмминга на Python

Кодер — это данные на своих местах и три XOR по строкам матрицы H .

```
H = [  
    [1, 0, 1, 0, 1, 0, 1],  
    [0, 1, 1, 0, 0, 1, 1],  
    [0, 0, 0, 1, 1, 1, 1],  
]  
PARITY = (0, 1, 3) # проверочные позиции 1, 2, 4  
  
def encode(data):  
    x = [0] * 7  
    x[2], x[4], x[5], x[6] = data # данные на позициях 3, 5, 6, 7  
    for i, row in zip(PARITY, H): # строка H обслуживает свою позицию  
        x[i] = sum(x[j] for j in range(7) if row[j] and j != i) % 2  
    return x
```

Декодер

Декодер считает синдром, читает в нём номер позиции и переворачивает бит.

```
def decode(r):  
    s = [sum(H[i][j] * r[j] for j in range(7)) % 2 for i in range(3)]  
    if any(s):  
        pos = s[0] + 2 * s[1] + 4 * s[2] # синдром называет позицию  
        print("синдром", s, "ошибка в позиции", pos)  
        r[pos - 1] ^= 1  
    return r[2], r[4], r[5], r[6]
```

Запуск с ошибкой

Шум моделируем одним XOR после кодирования.

```
x = encode([1, 0, 1, 1])  
x[4] ^= 1 # канал перевернул пятый бит  
print(decode(x))
```

```
синдром [1, 0, 1] ошибка в позиции 5  
(1, 0, 1, 1)
```

Синдром (1, 0, 1) прочитался как число 5, перевёрнутый бит встал на место — данные вернулись.

CRC и контрольные суммы

Обнаруживающие механизмы вокруг нас решают более простую задачу.

- Контрольная сумма пакета складывает его слова: несовпадение сигнализирует о порче.
- CRC делит пакет на порождающий полином: нулевой остаток означает целостность.
- Ни тот, ни другой не знает позицию ошибки, поэтому исправить её не может.

Обнаружение требует $d \geq s + 1$, исправление — $d \geq 2t + 1$: коррекция вдвое дороже.

Поэтому в сети ставят CRC и просят повторную передачу, а коды Хэмминга живут в памяти, где переспроса нет.

Что мы поняли: коды

Кодовое расстояние задаёт способность кода, а границы — цену защиты.

- Кодирование — инъекция: два сообщения с одним словом декодер не различит.
- Расстояние Хэмминга — метрика, а кодовое расстояние d обнаруживает до $d - 1$ и исправляет до $\lfloor \frac{d-1}{2} \rfloor$ ошибок.
- Код Хэмминга $(7, 4)$: проверочная матрица собирает группы, синдром называет позицию ошибки, а $16 \cdot 8 = 2^7$ — код совершенный.
- Граница Хэмминга считает шары, граница Синглтона $k \leq n - d + 1$ отделяет MDS-коды: над $\text{GF}(2)$ это чётность и повторение.
- За пределами гарантий две ошибки маскируются под одну, три проходят незаметно.
- Линейный код — подпространство в $\text{GF}(2)^n$: порождающая матрица кодирует, расстояние равно минимальному весу.

Вопросы для самопроверки

Вопросы для самопроверки

1. Вычислите $d(110010, 010101)$.
2. Почему код чётности не исправляет ни одной ошибки, хотя обнаруживает одну?
3. Закодируйте $(0, 1, 1, 0)$, внесите одиночную ошибку и исправьте её по синдрому.
4. Почему линейному коду достаточно минимального веса ненулевого слова?
5. Код чётности $(n, n - 1, 2)$ достигает границы Синглтона. Как это проверить?
6. Функция кодирования отправляет сообщения A и B в одно слово. Что нарушено и чем это грозит?
7. Дан код $\{000, 111\}$. Найдите его параметры $(n, |C|, d)$, скорость и проверьте границу Хэмминга.

Что почитать

- m13 — «Расстояние Хэмминга и геометрия кода», «Код Хэмминга», «Линейные коды и $GF(2)$ ».